

Week 2 Assignment Report

Parallel Thinking & CUDA Programming Basics

Priyanshu Gonnade

Roll Number: 24B3959

January 2, 2026

1 Overview and Execution Flow (CPU → GPU)

In this assignment, we implemented and analyzed a basic CUDA kernel for vector addition. The goal was to understand how computation is offloaded from the CPU (host) to the GPU (device), and how grid and block configurations affect correctness and scalability.

The execution flow is as follows:

1. Host-side preparation:

- Allocate memory for input and output vectors on the CPU.
- Initialize input vectors with test values.

2. Device memory allocation:

- Allocate GPU memory using `cudaMalloc` for input and output arrays.

3. Memory transfer (Host → Device):

- Copy input data from host to device using `cudaMemcpy`.

4. Kernel launch:

- Launch the CUDA kernel with a chosen grid size and block size.
- Each GPU thread computes one element of the output vector.

5. Bounds checking inside kernel:

- Each thread checks whether its global index is within the valid input range.

6. Synchronization:

- `cudaDeviceSynchronize()` is used to ensure kernel execution completes.

7. Memory transfer (Device → Host):

- Copy the result back to the CPU.

8. Verification:

- GPU results are compared with a CPU reference implementation to verify correctness.

2 Grid and Block Configuration Results

The grid size is computed as:

$$\text{gridDim.x} = \left\lceil \frac{n}{\text{blockDim.x}} \right\rceil$$

The total number of threads launched is:

$$\text{Total Threads} = \text{gridDim.x} \times \text{blockDim.x}$$

2.1 Experimental Results

Table 1 presents the experimental results for different input sizes and block configurations.

Table 1: Performance results for vector addition kernel with varying input sizes and block sizes

Input Size (N)	Block Size	Grid Size	Total Threads	Kernel Time (ms)	Correct
1000	32	32	1024	0.197760	Yes
1000	128	8	1024	0.200032	Yes
1000	256	4	1024	0.210752	Yes
1000	512	2	1024	0.216096	Yes
100000	32	3125	100000	0.158976	Yes
100000	128	782	100096	0.160256	Yes
100000	256	391	100096	0.168576	Yes
100000	512	196	100352	0.185728	Yes
10000000	32	312500	10000000	1.483008	Yes
10000000	128	78125	10000000	0.858912	Yes
10000000	256	39063	10000128	0.858240	Yes
10000000	512	19532	10000384	0.824384	Yes

3 Effect of Input Size and Block Size

3.1 Effect of Input Size

- For small input sizes ($N = 10^3$), kernel launch overhead dominates, and block size has minimal impact.
- For medium input sizes ($N = 10^5$), GPU occupancy becomes important and performance differences emerge.
- For large input sizes ($N = 10^7$), the GPU is fully utilized, and performance is dominated by memory bandwidth and warp-level parallelism.

3.2 Effect of Block Size

- Smaller block sizes (32) lead to excessive scheduling overhead and poor latency hiding for large N .
- Medium block sizes (128–256) provide a good balance between occupancy and resource usage.
- Larger block sizes (512) perform best for simple kernels like vector addition due to reduced scheduling overhead.

3.3 Optimization Guideline

The block size should be chosen to maximize GPU occupancy without causing excessive register or shared memory pressure. Performance conclusions should always be drawn using sufficiently large input sizes.

4 Answers to Conceptual Questions

Q1: What happens if the total number of threads exceeds N ?

Threads with indices greater than or equal to N perform no useful work and must be prevented from accessing invalid memory.

Q2: Why is bounds checking required inside the kernel?

Bounds checking avoids out-of-range memory access, which can cause undefined behavior or program crashes.

Q3: Which configurations failed or behaved unexpectedly?

No configurations failed. All tested configurations produced correct results when bounds checking was applied.

Q4: When would you choose a smaller block size vs. a larger one?

Smaller block sizes may be preferred for very small workloads or kernels with high register usage, while larger block sizes are beneficial for simple, memory-bound kernels with large input sizes.